Faculty of Engineering and Technology

Department of Computer Science

Software Quality Assurance - COMP438

Assignment Report No.1

Prepared by:

Students Name:	**IDs:**
Ahmad Hamza	1210381
Ayah Abdalmuti	1212074

Instructor: Dr. Samer Zain.

Section: 1

Semester: 2025/2026.

Code Selection Sheet

1. Project Name: Human Resource Management System **(HR-Master).**

2. Links: **https://github.com/geniusuraj/HRMaster/tree/main**

3. Reason for Choosing it : We chose this project because it fits the requirements of the code inspection assignment. It is an open-source Java project available on GitHub, with a code size within the required range and multiple interacting classes. The project has sufficient functionality to identify common coding errors, evaluate code structure and documentation, and analyze maintainability issues. This makes it suitable for performing a formal code inspection.

4. Code Size (LOC):

File	**LOC**
Department.java	56
Employee.java	90
LeaveRequest.java	88
Admin.java	94
EmployeeService.java	80

The overall sum of LOC is 408.

5. Description of the Functionality: HR-Master is a simple Human Resource Management System that supports basic HR operations. The system allows administrators to manage departments, register employees, handle leave requests by approving or denying them, and remove employees from the system. Employees can log in, submit leave requests, view the status and history of their leaves, update their account information, and check their salary details.

6. Screenshot of Repository Commit History:

See attached screenshot of GitHub commit history.

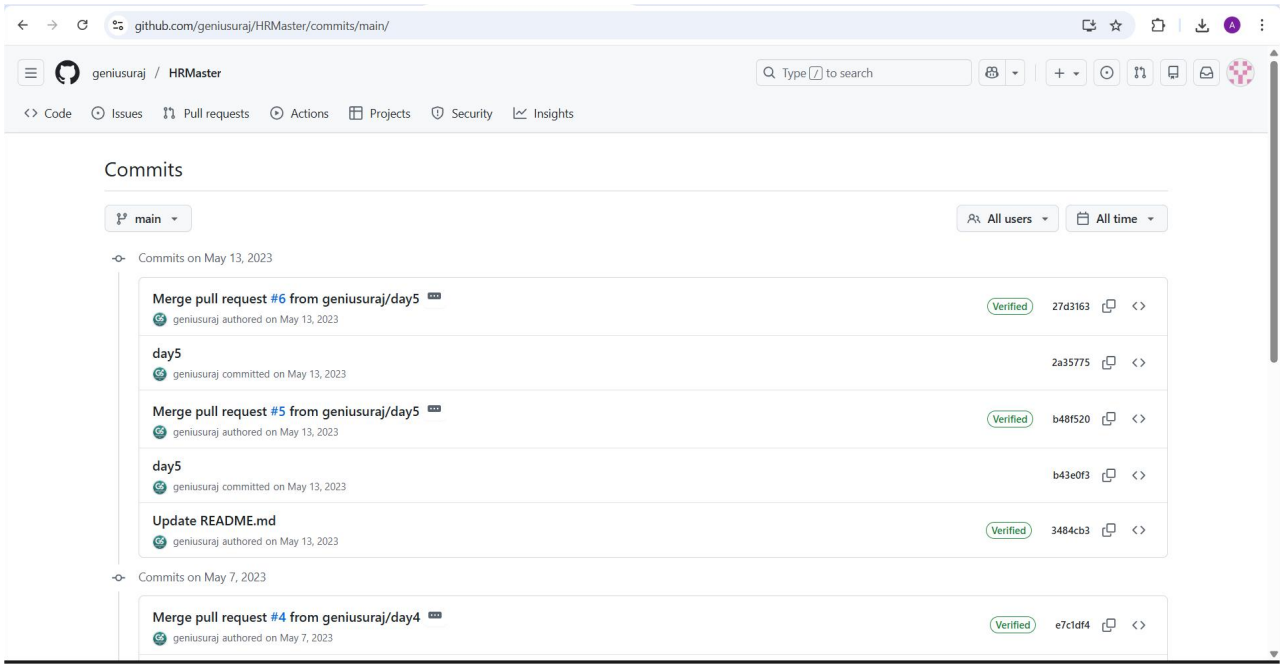

Figure 1: GitHub commit history.

Inspection Report

1. Introduction

Brief description of component:

HR-Master is a Java-based Human Resource Management System that helps manage basic HR tasks. Administrators can add and manage departments, register employees, handle leave requests by approving or denying them, and remove employees when needed. Employees can log in, submit leave requests, check their status and history, update account information, and view salary details. The system is built with multiple interacting classes.

Inspection goals selected:

In this inspection, we focused on common code errors like logic mistakes, null pointer exceptions, and missing error handling. We also examined code modularity and structure, looking at cohesion, coupling and long methods. Documentation and comments were reviewed for completeness and accuracy. Additionally, we analyzed maintainability metrics, including lines of code, method lengths, and duplication, to evaluate how easy the code is to maintain, extend, and update.

2. Defect List Table

ID	***Category***	***Location (file/line)***	***Severity***	***Description***	***Suggested Fix***
1	Common Code Error	Employee.java / 29 , setEmail()	Medium	Method sets username instead of email. This may cause confusion and incorrect behavior.	Change this.username = email; to this.email = email;
2	Common Code Error	Admin.java / 60, registerNewEmployee ()	High	setDateOfJoining(new Date(departmentId)) is incorrect. It uses departmentId instead of current date.	Use employee.setDateOfJoining(new Date(System.currentTimeMillis()));

ID	*Category*	*Location (file/line)*	*Severity*	*Description*	*Suggested Fix*
3	Common Code Error	EmployeeService.java / 15 , login()	Medium	getSingleResult() throws exception if no match is found. No error handling implemented.	Wrap in try-catch or use getResultList() and check if empty
4	Code Smell	Admin.java / 71, changeDepartmentOf Employee()	Medium	Method directly sets department without checking if new department exists.	Check if new department is valid before setting
5	Code Smell	EmployeeService.java / 61 , getTotalSalaryForMo nth()	Low	Method ignores the month parameter; always returns salary.	Implement proper calculation based on month parameter
6	Code Smell	EmployeeService.java / 69, getAnnualSalaryForFi nancialYear()	Low	Method multiplies salary by 12 without considering start date or leaving months.	Consider joining date and leaving date for precise calculation
7	Maintainability Issue	Admin.java / 80, updateLeaveRequest()	Low	Status is set using integer (1 or 2) rather than enum or constant.	Use enum or constants for clarity
8	Documentation Issue	All Service Classes	Low	Methods have no Javadoc comments; difficult for new developers to understand	Add Javadoc for all public methods
9	Security Issue	Employee.java / 14, password field	High	Password stored in plain text	Implement password hashing before storing in database
10	Common Code Error	LeaveRequest.java / 57, getEndDate()	Low	No validation to ensure endDate is after startDate	Add validation in setter or during creation
11	Maintainability Issue	Department.java/ 49	Low	employees set is exposed via getter, allowing external modification	Return an unmodifiable copy or provide controlled methods

ID	*Category*	*Location (file/line)*	*Severity*	*Description*	*Suggested Fix*
12	Common Code Error	Employee.java / 86, setDateOfJoining()	Medium	Method is empty (TODO) and does not set the joining date.	Implement method to assign the correct joining date.
13	Common Code Error	Admin.java / 47, updateDepartmentName()	Medium	No check if department exists before updating; may cause NullPointerException.	Verify department exists before updating.
14	Common Code Error	Admin.java / 62, registerNewEmployee()	High	No exception handling on database persist; can fail silently or crash.	Wrap EntityManager persist in try-catch to handle exceptions.
15	Security Issue	Admin.java / 59, registerNewEmployee()	High	Hardcoded default password "123456" for new employees.	Prompt for password or generate secure random password, store hashed.
16	Common Code Error	LeaveRequest.java /68, 76, reason/status fields	Low	No validation for input length or correctness; can store invalid data.	Add input validation for text length and allowed values.
17	Code Smell / Maintainability	Admin.java / 61, & EmployeeService.java / 12	Medium	Tight coupling due to direct use of EntityManager in multiple classes.	Introduce DAO/repository layer to reduce coupling and improve testability.

3. Code Smells & Design Issues

Cohesion/Coupling analysis:

- **Admin.java** handles multiple tasks like login, department management, employee registration, and leave approvals. This reduces **cohesion** and violates the single-responsibility principle.

- **EmployeeService.java** mixes business logic and direct database access via EntityManager, increasing **coupling** and reducing maintainability.

Architecture/Design comments:

- Classes are generally organized, but some methods are long, reducing readability.
- Logic is duplicated between **Admin** and **EmployeeService**, such as leave handling and salary calculations. Refactoring these into **helper/util classes** would improve modularity.
- Documentation is sparse; adding Javadoc for public methods would improve maintainability.
- The architecture is simple and functional, but scalability and testability are limited due to long methods and tight coupling.

Additional Inspection Goals (required):

Maintainability Metrics Observed

1. Duplicated Logic: leave handling and salary calculation logic appear in multiple classes, which increases maintenance effort.

2. Coupling: direct use of **EntityManager** in multiple classes results in tight coupling and reduces testability.

4. Conclusion

Reflection on inspection process:

HRMaster is functional and covers basic HR management tasks, but it has several important issues: Long methods, high coupling, missing documentation, logic inconsistencies, and incomplete implementations (like: setDateOfJoining ( )). The inspection highlighted the importance of modular design, maintainable code, and proper documentation. Refactoring large methods, improving cohesion, reducing duplication, and adding exception handling and comments would significantly improve maintainability and reliability. Overall, the system is suitable for learning and code inspection exercises, and the issues identified provide clear guidance for improving code quality.